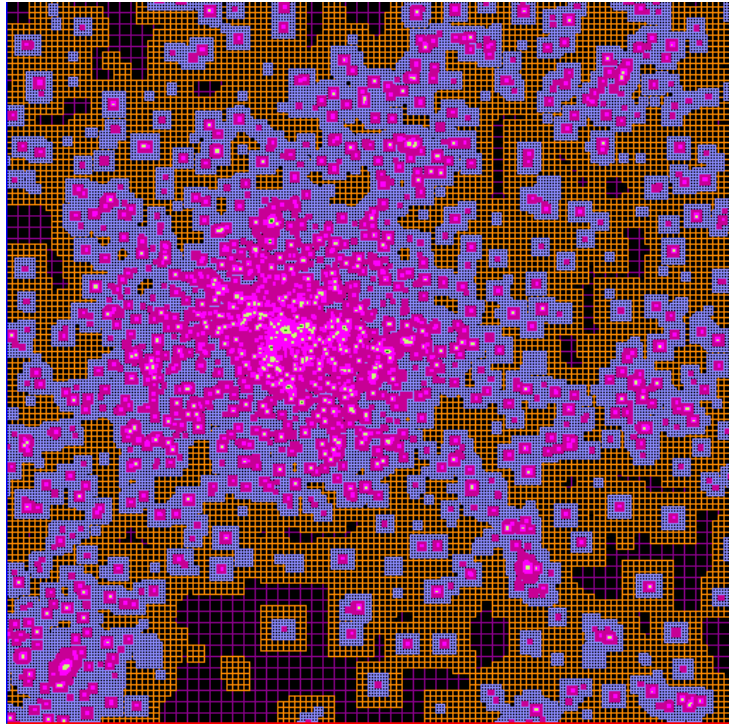




RAMSES User's Guide

Self-gravitating fluid dynamics
with Adaptive Mesh Refinement
using massively parallel computers.

I r f u

cea

saclay

Romain Teyssier

Contents

1	Introduction	4
1.1	About This Guide	4
1.2	Getting RAMSES	4
1.3	Main features	4
1.4	Acknowledgements	5
1.5	The CeCILL License	5
2	Getting started	6
2.1	Obtaining the package	6
2.2	Compiling the code	6
2.3	Executing the test case	7
2.4	Reading the Log File	8
3	Runtime Parameters	11
3.1	Updated physics namelist blocks (this fork)	13
3.2	Global parameters	14
3.3	AMR grid	15
3.4	Initial conditions	16
3.5	Output parameters	17
3.6	Movie parameters	18
3.7	Boundary conditions	19
3.8	Hydrodynamics solver	20
3.9	Physical parameters	21
3.10	Poisson solver	22
3.11	Refinement strategy	23
4	Cosmological simulations	24
4.1	Parameter file and initial conditions	24
4.2	Memory management	25
4.3	Restarting a run	25
4.4	Parallel computing	26
4.5	Post-processing utilities	28
4.6	Zoom simulations	29
5	Advanced simulations	30
5.1	Patching the code	30
5.2	Physical units	30
5.3	Initial conditions	30
5.4	Boundary conditions	31
5.5	External gravity sources	32
5.6	External thermal sources	32
6	Publication policy	33
	Index	34

1 Introduction

The RAMSES package is intended to be a versatile platform to develop applications using Adaptive Mesh Refinement for computational astrophysics. The current implementation allows solving the Euler equations in presence of self-gravity and cooling, treated as additional source terms in the momentum and energy equations. The RAMSES code can be used on massively parallel architectures when properly linked to the MPI library. It can also be used on single processor machines without MPI. Output files are generated using native RAMSES Fortran unformatted files. A suite of post-processing routines is delivered within the present release, allowing the user to perform a simple analysis of the generated output files.

1.1 About This Guide

The goal of this User's Guide is to provide a step-by-step tutorial in using the RAMSES code. This guide will first describe a simple example of its use. More complex set-up will be addressed at the end of the document. Typical RAMSES users can be grouped into 3 categories:

- **Beginners:** It is possible to execute RAMSES using only run parameter files. The code is compiled once, and the user only modifies the input file to perform various simulations. Cosmological simulations can be performed quite efficiently in this way, using the initial conditions generated by external packages such as `mpggrafic`.
- **Intermediate users:** For more complex applications, the user can easily modify a small set of routines in order to specify specific initial or boundary conditions. These routines are called “patches” and the code should be recompiled each time these routines are modified.
- **Advanced users:** It is finally possible to modify the base scheme, add new equations, or add new routines in order to modify the default RAMSES application. This guide will not describe these advanced features. In this case, a new documentation would be given separately.

1.2 Getting RAMSES

RAMSES software can be downloaded in the *Codes* section from various web sites, the most frequently updated ones being the Bitbucket repository <https://bitbucket.org/rteyssie/ramses> and <http://www.ics.uzh.ch/~teyssier/ramses>. It is freely distributed under the CeCILL software license (see section 1.5 and <http://www.cecill.info/>) according the French legal system *for non-commercial use only*. For commercial use of RAMSES, please contact the author, but be prepared for a massive financial compensation !

1.3 Main features

RAMSES contains various algorithms designed for:

- Cartesian AMR grids in 1D, 2D or 3D
- Solving the Poisson equation with a Multi-grid and a Conjugate Gradient solver
- Using various Riemann solvers (Lax-Friedrich, HLLC, exact) for adiabatic gas dynamics
- Computing collision-less particles (dark matter and stars) dynamics using a PM code
- Computing the cooling and heating of a metal-rich plasma due to atomic physics processes and an homogeneous UV background (Haardt and Madau model).

- Implementing a model of star-formation based on a standard Schmidt law with the traditional set of parameters.
- Implementing a model of supernovae-driven winds based on a local Sedov blast wave solution.

All these features can be used and parameterized using the RAMSES parameter file, based on the Fortran “namelist” format.

1.4 Acknowledgements

The development of the RAMSES code has been initiated and coordinated by the author. The author would like to thank all collaborators who took an active role in the development of this version. They are cited in chronological order.

- Matthias Gonzalez and Dominique Aubert (initial conditions)
- Stéphane Colombi and Stéphanie Courty (cooling and atomic physics)
- Yann Rasera (star formation, post-processing)
- Yohan Dubois (supernovae feedback)
- Thomas Guillet (multigrid Poisson solver)
- Sébastien Fromang, Patrick Hennebelle and Emmanuel Dormy (MHD solver).
- Philippe Wautelet and Philippe Sériès (code optimization)

I would like to thank my collaborators for helping me developing more advanced versions of RAMSES, not yet available as complete releases, since it is mostly work in progress.

- Benoît Commerçon (thermal conduction)
- Édouard Audit and Dominique Aubert (radiative transfer)
- Rémi Abgrall and Richard Saurel (multifluid)

1.5 The CeCILL License

This software is under Copyright of CEA and its author, Romain Teyssier.

This software is governed by the CeCILL license under French law and abiding by the rules of distribution of free software. You can use, modify and/or redistribute the software under the terms of the CeCILL license as circulated by CEA, CNRS and INRIA at the following URL: <http://www.cecill.info/>.

As a counterpart to the access to the source code and rights to copy, modify and redistribute granted by the license, users are provided only with a limited warranty and the software’s author, the holder of the economic rights, and the successive licensors have only limited liability.

In this respect, the user’s attention is drawn to the risks associated with loading, using, modifying and/or developing or reproducing the software by the user in light of its specific status of free software, that may mean that it is complicated to manipulate, and that also therefore means that it is reserved for developers and experienced professionals having in-depth IT knowledge. Users are therefore encouraged to load and test the software’s suitability as regards their requirements in conditions enabling the security of their systems and/or data to be ensured and, more generally, to use and operate it in the same conditions as regards security.

The fact that you are presently reading this means that you have had knowledge of the CeCILL license and that you accept its terms.

2 Getting started

In this section, we will explain step by step how to get the RAMSES package and install it, then how to perform a simple test to check the installation.

2.1 Obtaining the package

The package can be downloaded from the Bitbucket repository <https://bitbucket.org/rteyssie/ramses> using the command

```
git clone https://bitbucket.org/rteyssie/ramses
```

You will get a clone of the git repository, with a directory named `trunk/ramses/`. In this directory, you will find the following directory list:

README	aton/	doc/	mhd/	pario/	pm/	rt/
amr/	bin/	hydro/	namelist/	patch/	poisson/	utils/

Each directory contains a set of files with a given common purpose. For example, `amr/` contains all F90 routines dealing with the AMR grid management and MPI communications, while `hydro/` obviously contains all F90 routines dealing with hydrodynamics. The first directory you are interested in is the `bin/` directory, in which the code will be compiled.

2.2 Compiling the code

In this `bin/` directory, you will find a Makefile. The first thing to do is to edit the Makefile and modify the two variables `F90` and `FFLAGS`. Several examples corresponding to different Fortran compilers are given. The default values are:

```
F90 = gfortran
FFLAGS = -x f95-cpp-input $(DEFINES) -DWITHOUTMPI
```

The first variable is obviously the command used to invoke the Fortran compiler. In this case, this is the GNU Fortran compiler. The second variable contains Fortran flags and preprocessor directives. The last directive, `-DWITHOUTMPI`, when used, switches off all MPI routines. On the other hand, if you don't use this directive, the code must be linked to the MPI library. We will discuss this point later. These directives are called *Compilation Time Parameters*. They should be defined within the Makefile. Default values are:

```
NDIM=1
SOLVER=hydro
```

The first variable, `NDIM`, sets the dimensionality of the problem. The default value is for 1D, plan-parallel flows. Allowed values are 1, 2 and 3. The second directive defines the solver, which in the current release of RAMSES can be `hydro` or `mhd`. Other solvers are currently under development, such as a relativistic solver `rhd`, or a multifluid solver `multimat` and so on.

There are 3 other preprocessor directives that can be use in RAMSES: `-DNVAR=NDIM+2`, useful to set more variables in the hydro solver, `-DNPRES=8`, to set the number of bytes used for real numbers. `NPRES=8` corresponds to double precision arithmetic and `NPRES=4` to single precision. This option is useful to save memory during memory intensive runs. Finally, you can use `-DNVECTOR=64` to set the size of the vector sweeps for computationally intensive operations.

To compile RAMSES, execute:

```
$ make
```

If everything goes well, all source files will be compiled and linked into an executable called `ramses1d`.

2.3 Executing the test case

To test the compilation, you need to execute a simple test case. Go up one level and type the following command:

```
$ bin/ramses1d namelist/tube1d.nml
```

The first part of the command is the executable, and the second part, the only command line argument, is an input file containing all *Run Time Parameters*. Several examples of such parameter files are given in the `namelist/` directory. The run we have just performed, `tube1d.nml`, is the Sod's test, a simple shock tube simulation in 1D. For comparison, we now show the last 14 lines of standard output:

```
1428 Mesh structure
1429 Level 1 has      1 grids (      1,      1,      1,)
1430 Level 2 has      2 grids (      2,      2,      2,)
1431 Level 3 has      4 grids (      4,      4,      4,)
1432 Level 4 has      8 grids (      8,      8,      8,)
1433 Level 5 has     16 grids (     16,     16,     16,)
1434 Level 6 has     27 grids (     27,     27,     27,)
1435 Level 7 has     37 grids (     37,     37,     37,)
1436 Level 8 has     17 grids (     17,     17,     17,)
1437 Level 9 has     16 grids (     16,     16,     16,)
1438 Level 10 has    13 grids (     13,     13,     13,)
1439 Main step=      43 mcons=-1.97E-16 econs= 1.61E-16 epot= 0.00E+00 ekin= 1.38E+00
1440 Fine step=    688 t= 2.45047E-01 dt= 3.561E-04 a= 1.000E+00 mem= 7.6%
1441 Run completed
```

To save the standard output in a file, the user is encouraged to redirect the standard output in a *Log File*, in which all control variables are outputted and stored, as well as simulation data for 1D cases only

```
$ bin/ramses1d namelist/tube1d.nml > tube1d.log
```

To monitor the progress of longer runs, you can also redirect standard output to both a log file and the terminal at the same time with:

```
$ bin/ramses1d namelist/tube1d.nml | tee tube1d.log
```

2.4 Reading the Log File

We will now briefly describe the structure and the nature of the information available in the Log Files. We will use as example the file `tube1d.log`, which should contain, starting from the top

```

1  _/_/_/      _/_/      _/      _/      _/_/_/      _/_/_/_/      _/_/_/
2  _/      _/      _/      _/      _/_/_/_/      _/      _/      _/      _/
3  _/      _/      _/      _/      _/      _/      _/      _/      _/
4  _/_/_/      _/_/_/_/      _/      _/      _/_/      _/_/_/      _/_/
5  _/      _/      _/      _/      _/      _/      _/      _/      _/
6  _/      _/      _/      _/      _/      _/      _/      _/      _/
7  _/      _/      _/      _/      _/      _/      _/_/_/      _/_/_/_/      _/_/_/
8                                     Version 3.0
9                                     written by Romain Teyssier (University of Zurich)
10                                    (c) CEA 1999-2007, UZH 2008-2014
11
12 Working with nproc =      1 for ndim = 1
13 Using the hydro solver with nvar =  5
14
15 Building initial AMR grid
16 Initial mesh structure
17 Level  1 has          1 grids (          1,          1,          1,)
18 Level  2 has          2 grids (          2,          2,          2,)
19 Level  3 has          4 grids (          4,          4,          4,)
20 Level  4 has          8 grids (          8,          8,          8,)
21 Level  5 has          8 grids (          8,          8,          8,)
22 Level  6 has          8 grids (          8,          8,          8,)
23 Level  7 has          8 grids (          8,          8,          8,)
24 Level  8 has          8 grids (          8,          8,          8,)
25 Level  9 has          6 grids (          6,          6,          6,)
26 Level 10 has          4 grids (          4,          4,          4,)
27 Starting time integration
28 Output    58 cells
29 =====
30 lev      x          d          u          P
31  4  3.12500E-02  1.000E+00  0.000E+00  1.000E+00
32  4  9.37500E-02  1.000E+00  0.000E+00  1.000E+00
33
34      :
35
36  4  9.06250E-01  1.250E-01  0.000E+00  1.000E-01
37  4  9.68750E-01  1.250E-01  0.000E+00  1.000E-01
38 =====
39 Fine step=      0 t= 0.00000E+00 dt= 6.603E-04 a= 1.000E+00 mem= 3.2%
40 Fine step=      1 t= 6.60250E-04 dt= 4.420E-04 a= 1.000E+00 mem= 3.2%

```

After the code banner and copyrights, the first line indicates that you are currently using 1 processor and 1 space dimension for this run. The code then reports that it is building the initial AMR grid. The next lines give the current mesh structure.

The first level of refinement in RAMSES covers the whole computational domain with 2, 4 or 8 cells in 1, 2 or 3 space dimensions. The grid is then further refined up to `levelmin`, which

is this case is defined in the parameter file to be `levelmin=3`. The grid is then further refined up to `levelmax`, which is in this case `levelmax=10`. Each line in the Log File indicates the number of octs (or grids) at each level of refinement. The maximum number of grids in each level l is equal to 2^{l-1} in 1D, to 4^{l-1} in 2D and to 8^{l-1} in 3D. The numbers inside parentheses give the minimum, maximum and average number of grids per processor. This is obviously only relevant to parallel runs.

The code then indicates the time integration starts. After outputting the initial conditions to screen, the first *Control Line* appears, starting with the words `Fine step=`. The Control Line gives information on each *Fine Step*, its current number, its current time coordinate, its current time step. Variable `a` is for cosmology runs only and gives the current expansion factor. The last variable is the percentage of allocated memory currently used by RAMSES to store each flow variable on the AMR grid and to store each collision-less particle, if any.

In RAMSES, adaptive time stepping is implemented, which results in defining *Coarse Steps* and *Fine Steps*. Coarse Steps correspond to the coarse grid, which is defined by variable `levelmin`. Fine Steps correspond to finer levels, for which the time step has been recursively subdivided by a factor of 2. Fine levels are “sub-cycled” twice as more as their parent coarse level. This explains why, at the end of the Log File, only 43 Coarse Steps are reported (1 through 43), for 689 Fine Steps (numbered 0 to 688).

When a Coarse Step is reached, the code writes in the Log File the current mesh structure. A new Control Line then appears, starting with the words `Main step=`. This Control Line gives information on each Coarse Step, namely its current number, the current error in mass conservation within the computational box `mcons`, the current error in total energy conservation `econs`, the gravitational potential energy and the fluid total energy (kinetic plus thermal).

This constitutes the basic information contained in the Log File. In 1D simulations, output data are also written to standard output, and thus to the Log File. For 2D and 3D, output data are stored into unformatted Fortran binary files. 1D data are shown using 5 columns (level of refinement, position of the cell, density, velocity and pressure) as in the following Sod’s test example:

```

1282 | Output    142 cells
1283 | =====
1284 | lev      x      d      u      P
1285 |  5  1.56250E-02  1.000E+00  0.000E+00  1.000E+00
1286 |  5  4.68750E-02  1.000E+00  0.000E+00  1.000E+00
1287 |  5  7.81250E-02  1.000E+00  0.000E+00  1.000E+00
1288 |  5  1.09375E-01  1.000E+00  1.564E-09  1.000E+00
1289 |  6  1.32812E-01  1.000E+00  2.112E-08  1.000E+00

```

You can cut and paste the 142 lines into another file and use your favorite data viewer like `xmgrace` or `gnuplot` to visualize the results. These should be compared to the plots shown on Figure 1. If you have obtained comparable numerical values and levels of refinements, your installation is likely to be valid. You are encouraged to edit the parameter file `tube1d.log` and play around with other parameter values, in order to test the code performances. You can also use other Parameter Files in the `namelist/` directory.



Do not forget to recompile entirely the code (`make clean`, then `make`) with `NDIM=2` for 2D cases like `sedov2d.nml` or `NDIM=3` for 3D cases such as `sedov3d.nml`.

In the next section, we will describe in more detail the various Runtime Parameters available within RAMSES.

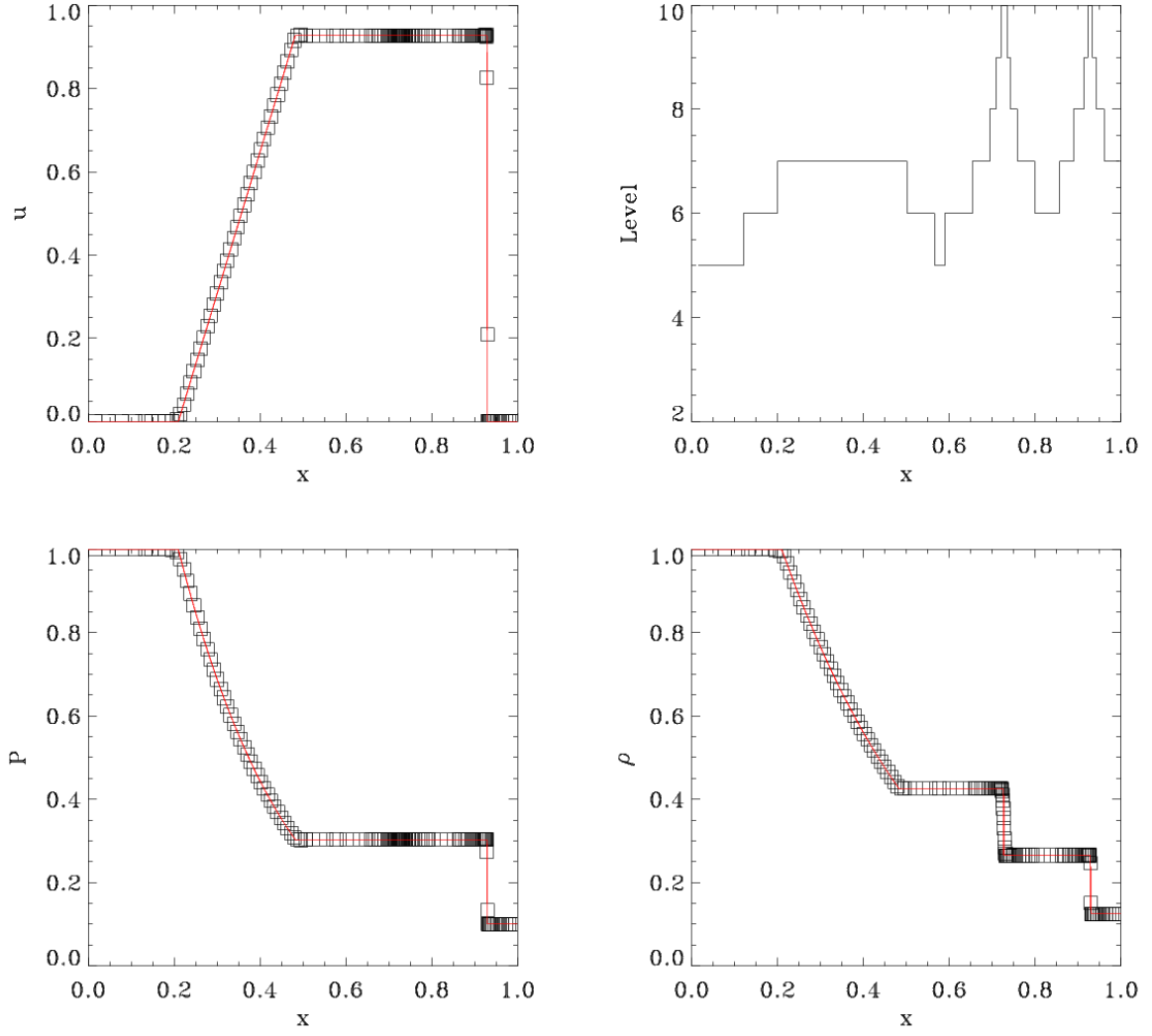


Figure 1: Numerical results obtained with RAMSES for the Sod shock tube test (symbols) compared to the analytical solution (red line).

3 Runtime Parameters

The RAMSES parameter file is based on the Fortran namelist syntax. The Sod test parameter file is shown below, as it should appear if you edit it.

```
1 This is the RAMSES parameter file for Sod's shock tube test.
2
3 &RUN_PARAMS
4 hydro=.true.
5 nsubcycle=3*1,2
6 /
7
8 &AMR_PARAMS
9 levelmin=3
10 levelmax=10
11 ngridmax=2000
12 nexpanse=1
13 boxlen=1.0
14 /
15
16 &BOUNDARY_PARAMS
17 nboundary=2
18 ibound_min=-1,+1
19 ibound_max=-1,+1
20 bound_type= 1, 1
21 /
22
23 &INIT_PARAMS
24 nregion=2
25 region_type(1)='square'
26 region_type(2)='square'
27 x_center=0.25,0.75
28 length_x=0.5,0.5
29 d_region=1.0,0.125
30 u_region=0.0,0.0
31 p_region=1.0,0.1
32 /
33
34 &OUTPUT_PARAMS
35 noutput=1
36 tout=0.245
37 /
38
39 &HYDRO_PARAMS
40 gamma=1.4
41 courant_factor=0.8
42 slope_type=2
43 scheme='muscl'
44 /
45
46 &REFINE_PARAMS
```

```

47 err_grad_d=0.05
48 err_grad_u=0.05
49 err_grad_p=0.05
50 interpol_var=0
51 interpol_type=2
52 /

```

This parameter file is organized in namelist blocks. Each block starts with `&BLOCK_NAME` and ends with the character “/”. Within each block, you can specify parameter values using standard Fortran namelist syntax. There are currently 9 different parameter blocks implemented in RAMSES.



4 parameter blocks are mandatory and must always be present in the parameter file. These are `&RUN_PARAMS`, `&AMR_PARAMS`, `&OUTPUT_PARAMS` and `&INIT_PARAMS`.

The 5 other blocks are optional. They must be present in the file only if they are relevant to the current run. These are `&BOUNDARY_PARAMS`, `&HYDRO_PARAMS`, `&PHYSICS_PARAMS`, `&POISSON_PARAMS` and finally `&REFINE_PARAMS`. We now describe each parameter block in more detail.

3.1 Updated physics namelist blocks (this fork)



In this codebase, the old `&PHYSICS_PARAMS` block is deprecated and explicitly rejected at runtime. The updated block names are `&COOLING_PARAMS`, `&SF_PARAMS`, `&FEEDBACK_PARAMS` and `&UNITS_PARAMS`.

The new blocks are parsed in `hydro/read_hydro_params.f90`, and are used by the modules listed below:

- `&COOLING_PARAMS`: cooling/heating and UV background (`hydro/cooling_module.f90`).
- `&SF_PARAMS`: star formation thresholds and models (`pm/star_formation.f90`).
- `&FEEDBACK_PARAMS`: supernova and mechanical feedback (`pm/feedback.f90`, `pm/mechanical_fine.f90`).
- `&UNITS_PARAMS`: code unit conversions (`amr/units.f90`).

In addition, two blocks used by this fork are handled outside the hydro parameter reader:

- `&SINK_PARAMS`: sink/SMBH/AGN controls (`pm/sink_particle.f90`).
- `&CLUMPFIND_PARAMS`: clump finder thresholds (`pm/clump_finder.f90`).

3.2 Global parameters

This block, called `&RUN_PARAMS`, contains the run global control parameters. These parameters are now briefly described. More thorough explanations will be given in dedicated sections.

Variable name, syntax and default value	Fortran type	Description
<code>cosmo=.false.</code>	Logical	Activate cosmological “super-comoving coordinates” system and expansion factor computing.
<code>pic=.false.</code>	Logical	Activate Particle-In-Cell solver
<code>poisson=.false.</code>	Logical	Activate Poisson solver.
<code>hydro=.false.</code>	Logical	Activate hydrodynamics or MHD solver.
<code>verbose=.false</code>	Logical	Activate verbose mode
<code>nrestart=0</code>	Integer	Output file number from which the code loads backup data and resumes the simulation. The default value, zero, is for a fresh start from the beginning. You should use the same number of processors than the one used during the previous run.
<code>nstepmax=1000000</code>	Integer	Maximum number of coarse time steps.
<code>ncontrol=1</code>	Integer	Frequency of screen output for Control Lines (to standard output) into the Log File).
<code>nremap=0</code>	Integer	Frequency of calls, in units of coarse time steps, for the load balancing routine, for MPI runs only, the default value, zero, meaning “never”.
<code>ordering='hilbert'</code>	Character LEN=128	Cell ordering method used in the domain decomposition of the grid among the processors, for MPI runs only. Possible values are <code>hilbert</code> , <code>planar</code> and <code>angular</code> .
<code>nsubcycle=2,2,2,2,2,</code>	Integer array	Number of fine level sub-cycling steps within one coarse level time step. Each value corresponds to a given level of refinement, starting from the coarse grid defined by <code>levelmin</code> , up to the finest level defined by <code>levelmax</code> . For example, <code>nsubcycle(1)=1</code> means that <code>levelmin</code> and <code>levelmin+1</code> are synchronized. To enforce single time stepping for the whole AMR hierarchy, you need to set <code>nsubcycle=1,1,1,1,1,</code>

3.3 AMR grid

This set of parameters, called `&AMR_PARAMS`, controls the AMR grid global properties. Parameters specifying the refinement strategy are described in the `&REFINE_PARAMS` block, which is used only if `levelmax>levelmin`.

Variable name, syntax and default value	Fortran type	Description
<code>levelmin=1</code>	Integer	Minimum level of refinement. This parameter sets the size of the coarse (or base) grid by $n_x = 2^{\text{levelmin}}$
<code>levelmax=1</code>	Integer	Maximum level of refinement. If <code>levelmax=levelmin</code> , this corresponds to a standard cartesian grid.
<code>ngridmax=0</code>	Integer	Maximum number of grids (or ocs) that can be allocated during the run <i>within each MPI process</i> .
<code>ngridtot=0</code>	Integer	Maximum number of grids (or ocs) that can be allocated during the run <i>for all MPI processes</i> . One has in this case <code>ngridmax=ngridtot/ncpu</code> .
<code>npartmax=0</code>	Integer	Maximum number of particles that can be allocated during the run <i>within each MPI process</i> .
<code>nparttot=0</code>	Integer	Maximum number of particles that can be allocated during the run <i>for all MPI processes</i> . Obviously, one has in this case <code>npartmax=nparttot/ncpu</code> .
<code>nexpand=1</code>	Integer	Number of mesh expansions (mesh smoothing).
<code>boxlen=1.0</code>	Real	Box size in user units

3.4 Initial conditions

This namelist block, called `&INIT_PARAMS`, is used to set up the initial conditions.

Variable name, syntax and default value	Fortran type	Description
<code>nregion=1</code>	Integer	Number of independent regions in the computational box used to set up initial flow variables.
<code>region_type='square'</code>	Character LEN=10 array	Geometry defining each region. square defines a generalized ellipsoidal shape, while point defines a delta function in the flow.
<code>x_center=0.0</code> <code>y_center=0.0</code> <code>z_center=0.0</code>	Real arrays	Coordinates of the center of each region.
<code>length_x=0.0</code> <code>length_y=0.0</code> <code>length_z=0.0</code>	Real arrays	Size in all directions of each region.
<code>exp_region=2.0</code>	Real array	Exponent defining the norm used to compute distances for the generalized ellipsoid. exp_region=2 corresponds to a spheroid, exp_region=1 to a diamond shape, exp_region>=10 to a perfect square.
<code>d_region=0.0</code> <code>u_region=0.0</code> <code>v_region=0.0</code> <code>w_region=0.0</code> <code>p_region=0.0</code>	Real arrays	Flow variables in each region (density, velocities and pressure). For point regions, these variables are used to defined extensive quantities (mass, velocity and specific pressure).
<code>filetype='ascii'</code>	Character LEN=20	Type of initial conditions file for particles. Possible choices are 'ascii' or 'grafic' .
<code>aexp_ini=10.0</code>	Real	This parameter sets the starting expansion factor for cosmology runs only. Default value is read in the IC file ('grafic' or 'ascii').
<code>multiple=.false.</code>	Logical	If .true. , each processor reads its own IC file ('grafic' or 'ascii'). For parallel runs only.
<code>initfile=' '</code>	Character LEN=80 array	Directory where IC files are stored. See section 4.1 for details.

3.5 Output parameters

This namelist block, called `&OUTPUT_PARAMS`, is used to set up the frequency and properties of data output to disk.

Variable name, syntax and default value	Fortran type	Description
<code>noutput=1</code>	Integer	Number of specified output time. At least one output time should be given, corresponding to the end of the simulation.
<code>tout=0.0,0.0,0.0,</code>	Real array	Value of specified output time.
<code>aout=1.1,1.1,1.1,</code>	Real array	Value of specified output expansion factor (for cosmology runs only). <code>aout=1.0</code> means “present epoch” or “zero redshift”.
<code>foutput=1000000</code>	Integer	Frequency of additional outputs in units of coarse time steps. <code>foutput=1</code> means one output at each time step. Specified outputs (see above) will not be superceded by this parameter.

3.6 Movie parameters

Besides the regular RAMSES outputs it is possible to produce snapshot projections of chosen variables, which can be then further used to produce movies/animations. The namelist that controls behaviour of snapshots is called `&MOVIE_PARAMS`.

Variable name, syntax and default value	Fortran type	Description
<code>movie=.false.</code>	Logical	Turns movie routine on/off.
<code>tendmov=0.0</code>	Real	The time of last snapshot.
<code>imovout=0</code>	Integer	Total number of snapshots.
<code>imov=1</code>	Integer	Number of the first snapshot.
<code>nw_frame=512</code>	Integer	Width of frame map.
<code>nh_frame=512</code>	Integer	Height of frame map.
<code>levelmax_frame=0</code>	Integer	Maximum level of refinement used during production of projection.
<code>xcentre_frame=0.0</code> <code>ycentre_frame=0.0</code> <code>zcentre_frame=0.0</code>	Real arrays	Sets centre of the box used for projection. E.g. if the box size is 20x20x20, then for single projection it should be specified as follows to get the full box projection: <code>xcentre_frame=10.,0.,0.</code> and the same for <code>y</code> and <code>z</code> . Arguments 2-4 are used to set up a time-dependent camera trajectory (using a spline).
<code>deltax_frame=0.0</code> <code>deltay_frame=0.0</code> <code>deltaz_frame=0.0</code>	Real arrays	Sets length of the side of the box used for projection. E.g. if the projection box size should be 5x5x5, then for single projection it should be specified as follows to get the full box projection: <code>xcentre_frame=5.,0.</code> and the same for <code>y</code> and <code>z</code> . Argument 2-4 are used to set up a time-dependent camera field-of-view (using a spline).
<code>proj_axis='z'</code>	Character LEN=5	Defines projection axis (<i>x</i> , <i>y</i> or <i>z</i>) for the snapshot; up to 5 projections (and thus movies) are allowed. The same projection can be specified multiple times.
<code>movie_vars=0</code>	Integer array	Specifies which variables are going to be projected (with density weighting). The array must have length of <code>NVAR+1</code> . The encoding is as follows (names of saved variables in SMALL CAPS): 0: TEMPerature, 1: DENSity, 2: VX, 3: VY, 4: VZ, 5: PRESSure, 6: METALlicity, 7...: VAR7...

3.7 Boundary conditions

This namelist block, called `&BOUNDARY_PARAMS`, is used to set up boundary conditions on the current simulation. If this namelist block is absent, periodic boundary conditions are assumed. Setting up other types of boundary conditions in RAMSES is quite complex. The reader is invited to read the corresponding section. The default setting, corresponding to a periodic box should be sufficient in most cases. The strategy to set up boundary conditions is based on using “ghost regions” outside the computational domain, where flow variables are carefully specified in order to mimic the effect of the chosen type of boundary. Note that *the order in which boundary regions are specified in the namelist is very important*, especially for reflexive or zero gradient boundaries. See section 5.4 for more information on setting up such boundary conditions. Specific examples can be found in the `namelist/` directory of the package.

Variable name, syntax and default value	Fortran type	Description
<code>nboundary=1</code>	Integer	Number of ghost regions used to specify the boundary conditions.
<code>bound_type=0,0,0,</code>	Integer array	Type of boundary conditions to apply in the corresponding ghost region. Possible values are: <code>bound_type=0</code> : periodic, <code>bound_type=1</code> : reflexive, <code>bound_type=2</code> : outflow (zero gradients), <code>bound_type=3</code> : inflow (user specified).
<code>d_bound=0.0</code> <code>u_bound=0.0</code> <code>v_bound=0.0</code> <code>w_bound=0.0</code> <code>p_bound=0.0</code>	Real arrays	Flow variables in each ghost region (density, velocities and pressure). They are used only for inflow boundary conditions.
<code>ibound_min=0</code> <code>jbound_min=0</code> <code>kbound_min=0</code>	Integer arrays	Coordinates of the lower, left, bottom corner of each boundary region. Each coordinate lies between -1 and $+1$ in each direction (see figure 3 on page 32).
<code>ibound_max=0</code> <code>jbound_max=0</code> <code>kbound_max=0</code>	Integer arrays	Likewise, for the upper, right and upper corner of each boundary region.

3.8 Hydrodynamics solver

This namelist is called `&HYDRO_PARAMS`, and is used to specify runtime parameters for the Godunov solver. These parameters are quite standard in computational fluid dynamics. We briefly describe them now.

Variable name, syntax and default value	Fortran type	Description
<code>gamma=1.4</code>	Real	Adiabatic exponent for the perfect gas EOS.
<code>courant_factor=0.5</code>	Real	CFL number for time step control (less than 1).
<code>smallr=1d-10</code>	Real	Minimum density to prevent floating exceptions.
<code>smallc=1d-10</code>	Real	Minimum sound speed to prevent floating exceptions.
<code>riemann='llf'</code>	Character LEN=20	Name of the desired Riemann solver. Possible choices are 'exact', 'acoustic', 'llf', 'h11' or 'h11c' for the hydro solver and 'llf', 'h11', 'roe', 'h11d', 'upwind' and 'hydro' for the MHD solver.
<code>riemann2d='llf'</code>	Character LEN=20	Name of the desired 2D Riemann solver for the induction equation (MHD only). Possible choices are 'upwind', 'llf', 'roe', 'h11', and 'h11d'.
<code>niter_riemann=10</code>	Integer	Maximum number of iterations used in the exact Riemann solver.
<code>slope_type=1</code>	Integer	Type of slope limiter used in the Godunov scheme for the piecewise linear reconstruction: <code>slope_type=0</code> : First order scheme, <code>slope_type=1</code> : MinMod limiter, <code>slope_type=2</code> : MonCen limiter. <code>slope_type=3</code> : Multi-dimensional MonCen limiter. In 1D runs only, it is also possible to choose: <code>slope_type=4</code> : Superbee limiter <code>slope_type=5</code> : Ultrabee limiter
<code>pressure_fix=.false.</code>	Logical	Activate hybrid scheme (conservative or primitive) for high-Mach flows. Useful to prevent negative temperatures.

3.9 Physical parameters

This namelist, called `&PHYSICS_PARAMS`, is used to specify physical quantities used in cosmological applications (cooling, star formation and supernovae feedback). We briefly describe them now.

Variable name, syntax and default value	Fortran type	Description
<code>cooling=.false.</code>	Logical	Activate the cooling and/or heating source term in the energy equation.
<code>metal=.false.</code>	Logical	Activate metal enrichment, advection and cooling. In this case, the preprocessor directive <code>-DNVAR=6</code> should be added in the Makefile before the compilation.
<code>haardt_madau=.false.</code>	Logical	Use the UV background model of Haardt and Madau. Default value <code>.false.</code> corresponds to a simple analytical model with parameters <code>J21</code> and <code>a_spec</code> .
<code>J21=0.0</code>	Real	Normalization for the UV flux of the simple background model. Default means “no UV”.
<code>a_spec=1.0</code>	Real	Slope for the spectrum of the simple background model. Default value corresponds to a standard “quasars + OB stars” spectrum.
<code>z_ave=0.0</code>	Real	Average metallicity used in the cooling function, in case <code>metal=.false.</code>
<code>t_star=0.0</code>	Real	Star formation time scale in Gyr. Default value of zero means no star formation.
<code>n_star=0.1,</code> <code>del_star=200.0</code>	Real	Typical interstellar medium physical density or comoving overdensity, used as star formation density threshold and as EOS density scale.
<code>T2_star=0.0, g_star=1.6</code>	Real	Typical interstellar medium polytropic EOS parameters.
<code>eta_sn=0.0, yield=0.1</code>	Real	Mass fraction of newly formed stars that explode into supernovae. Default value of zero means no supernovae feedback.

3.10 Poisson solver

This namelist, `&POISSON_PARAMS`, is used to specify runtime parameters for the Poisson solver. It is used only if `poisson=.true.` or `pic=.true.`

Two different Poisson solvers are available in RAMSES: conjugate gradient (CG) and multigrid (MG). Unlike the CG solver, MG has an initialization overhead cost (at every call of the solver), but is much more efficient on very big levels with few “holes”. The multigrid solver is therefore used for all coarse levels.

In addition, MG can be used on refined levels in conjunction with CG. The parameter `cg_levelmin` selects the Poisson solver as follows:

- Coarse levels are solved with MG
- Refined levels with $l < \text{cg_levelmin}$ are solved with MG
- Refined levels with $l \geq \text{cg_levelmin}$ are solved with CG

Variable name, syntax and default value	Fortran type	Description
<code>gravity_type=0</code>	Integer	Type of gravity force. Possible choices are: <code>gravity_type=0</code> : self-gravity (Poisson solver) <code>gravity_type>0</code> : analytical gravity vector <code>gravity_type<0</code> : self-gravity plus additional analytical density profile
<code>epsilon=1d-4</code>	Real	Stopping criterion for the iterative Poisson solver: residual 2-norm should be lower than <code>epsilon</code> times the right hand side 2-norm.
<code>gravity_params=0.0, 0.0, 0.0, 0.0,</code>	Real array	Parameters used to define the analytical gravity field (routine <code>gravana.f90</code>) or the analytical mass density field (routine <code>rho_ana.f90</code>).
<code>cg_levelmin=999</code>	Integer	Minimum level from which the Conjugate Gradient solver is used in place of the Multigrid solver.

3.11 Refinement strategy

This namelist, `&REFINE_PARAMS`, is used to specify refinement parameters controlling the AMR grid generation and evolution during the course of the run. It is used only if `levelmax > levelmin`.

Variable name, syntax and default value	Fortran type	Description
<code>mass_sph=0.0</code>	Real	Quasi-Lagrangian strategy: <code>mass_sph</code> is used to set a typical mass scale. For cosmo runs, its value is set automatically.
<code>m_refine=-1.,-1.,-1.,</code>	Real array	Quasi-Lagrangian strategy: each level is refined if the baryons mass in a cell exceeds <code>m_refine(ilevel)*mass_sph</code> , or if the number of dark matter particles exceeds <code>m_refine(ilevel)</code> , whatever the mass is.
<code>jeans_refine=-1.,-1.,</code>	Real array	Jeans refinement strategy: each level is refined if the cell size exceeds the local Jeans length divided by <code>jeans_refine(ilevel)</code> .
<code>floor_d=1d-10, floor_u=1d-10, floor_p=1d-10</code>	Real	Discontinuity-based strategy: density, velocity and pressure floor below which gradients are ignored.
<code>err_grad_d=-1.0, err_grad_u=-1.0, err_grad_p=-1.0</code>	Real	Discontinuity-based strategy: density, velocity and pressure relative variations above which a cell is refined.
<code>x_refine=0.0,0.0,0.0, y_refine=0.0,0.0,0.0, z_refine=0.0,0.0,0.0,</code>	Real arrays	Geometry-based strategy: center of the refined region at each level of the AMR grid.
<code>r_refine=1d10,1d10, a_refine=1.0,1.0, b_refine=1.0,1.0, exp_refine=2.0,2.0,</code>	Real arrays	Geometry-based strategy: size and shape of the refined region at each level.
<code>interpol_var=0</code>	Integer	Variables used to perform interpolation (prolongation) and averaging (restriction). <code>interpol_type=0</code> : conservatives (ρ , ρu , ρE) <code>interpol_type=1</code> : primitives (ρ , ρu , ρe)
<code>interpol_type=1</code>	Integer	Type of slope limiter used in the interpolation scheme for newly refined cells. <code>interpol_type=0</code> : Straight injection (1 st order) <code>interpol_type=1</code> : MinMod limiter, <code>interpol_type=2</code> : MonCen limiter.

4 Cosmological simulations

In this section, we describe in more detail how RAMSES can be used to perform cosmological simulations. Useful concepts related to parallel computing or post-processing will be introduced, and can also be used for non-cosmological runs. Cosmological simulations are performed by specifying `cosmo=.true.` in the `&RUN_PARAMS` namelist.

4.1 Parameter file and initial conditions

The first thing to do when performing cosmological simulations is to generate initial conditions as Gaussian random fields. The easiest way is to use the freely available `grafic2` code, developed by Edmund Bertschinger at MIT (see <http://web.mit.edu/edbert>) or its parallel version `mpgrafic` developed by Christophe Pichon and Simon Prunet at IAP (see <http://www.projet-horizon.fr/>). These codes will generate initial conditions according to a given cosmological model and for a given box size. As outputs, 7 files will be generated, called `ic_deltab`, `ic_velcx`, `ic_velcy`, `ic_velcz`, `ic_velbx`, `ic_velby` and `ic_velbz`. The directory in which these files are stored should be entered in the Parameter File as parameter `initfile(1)` in namelist `&INIT_PARAMS`. Index 1 stands here for `levelmin`, and corresponds to the coarse grid. RAMSES will automatically read the cosmological parameters and the physical box length contained in these initial conditions files. The so-called “super-comoving” coordinate system is used in RAMSES for cosmological runs (see Martell & Shapiro 2003). If necessary, the translation from this scale-free coordinate system (`boxlen=1.0`) to the *cgs* system is performed using scaling factors stored in the output files. The units are set in routine `units.f90` in directory `amr/`.

The following namelist can be found in directory `namelist/` in the RAMSES package as file `cosmo.nml`. It is the Parameter File for a pure N -body simulation, using 128^3 particles and a 128^3 coarse grid with 7 additional levels of refinement. To specify that initial conditions are to be read in `grafic` files, `filetype='grafic'` should be set in namelist `&INIT_PARAMS`.

```
1 &RUN_PARAMS
2 cosmo=.true.
3 pic=.true.
4 poisson=.true.
5 nrestart=0
6 nremap=10
7 nsubcycle=1,2
8 ncontrol=1
9 /
10
11 &OUTPUT_PARAMS
12 foutput=10
13 noutput=10
14 aout=0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0
15 /
16
17 &INIT_PARAMS
18 filetype='grafic'
19 initfile(1)='/scratchdir/grafic_files'
20 /
21
22 &AMR_PARAMS
23 levelmin=7
```



```

24 levelmax=14
25 ngridtot=2000000
26 nparttot=3000000
27 nexpand=1
28 /
29
30 &REFINE_PARAMS
31 m_refine=7*8.
32 /

```

Parameters `ngridtot` and `nparttot` specify the maximum memory allocated for AMR grids and collisionless particles respectively. These numbers should be greater than or equal to the actual number of AMR grids and particles used during the course of the simulation.

`ngridtot` stands for the total number of AMR grids allocated over all MPI processes. The `ngridmax` parameter can be used equivalently, but stands for the local number of AMR grids within each MPI process. Obviously, one has `ngridtot=ngridmax*ncpu`.



Recall that, in RAMSES, we call “AMR grid” or “oct” a group of 2^{ndim} cells. If for some reason, during the course of the execution, the maximum allowed number of grids or particles has been reached, the simulation stops with the message:

```

No more free memory
Increase ngridmax

```

In this case, don't panic: just increase `ngridmax` in the Parameter File and resume the execution, starting from the last backup file.

4.2 Memory management

These two parameters control the memory allocated by RAMSES. It is important to know how much memory in Gb is actually allocated by RAMSES for a given choice of parameters. This can be approximated by:

- $0.7(\text{ngridmax}/10^6) + 0.7(\text{npartmax}/10^7)$ Gbytes for pure N -body runs,
- $1.4(\text{ngridmax}/10^6) + 0.7(\text{npartmax}/10^7)$ Gb for N -body and hydro runs,
- $1.0(\text{ngridmax}/10^6)$ Gb for pure hydro runs.

Because of MPI communications overheads, the actual memory used can be slightly higher. Note that these numbers are valid for double precision arithmetic. For single precision runs, using the preprocessor directive `-DNP=4`, you can decrease these figures by 40%.

4.3 Restarting a run

As we just discussed, it is possible to resume a RAMSES run if the execution has stopped abnormally. For that, RAMSES uses its output files, stored in directories called

```

output_00001/
output_00002/
output_00003/
output_00004/

```

Each directory contains all the necessary information for RAMSES to resume the execution. The frequency at which these output files are created is specified by parameter `foutput`, in units of coarse time steps. If you want to resume the execution starting from output directory number 4, you need to specify the corresponding number in parameter `nrestart=4`. If you set `nrestart=0`, the run will start from the beginning as a completely new run.



When restarting a job, you can change almost all run parameters. There are however some notable exceptions: The number of output times can only be increased, and only new output times can be added after the old ones. The number of processors used with MPI cannot change. If you want to change the number of processes, you should start from the very beginning.

4.4 Parallel computing

We are now ready to address the complex issue of parallel computing with RAMSES. It is based on the MPI library through regular calls of MPI communication routines. In order to compile and link RAMSES with the MPI library, you need first to remove the preprocessor directive `-DWITHOUTMPI` from the compilation options in the Makefile. Don't forget to type `make clean` and then `make` again.

In order to launch a parallel RAMSES run, type for example

```
$ mpirun -np 4 bin/ramses3d namelist/sedov3d.nml
```

The two key parameters for parallel runs are `nremap` and `ordering`, both contained in the `&RUN_PARAMS` namelist block. The first one specifies the frequency at which the code will optimize load balancing by adjusting the domain decomposition, in units of coarse time step. Since it is a rather costly operation, this should not be performed too frequently. On the other hand, if the AMR grid evolves significantly with time, the computational and memory load might be very inhomogeneous across the processors. The optimal choice for parameter `nremap` is therefore application-dependent. Bear in mind that if `nremap>10`, the associated overhead should be negligible.

The other important parameter for an efficient parallel computing strategy is `ordering`. This character string specifies the type of domain decomposition you want to use. There are 3 possible choices in RAMSES currently implemented: `'hilbert'` (default value), `'planar'` and `'angular'`. Each cell in RAMSES is ordered with an integer index according to a given space ordering. One of the most famous ordering used in computer science is the Peano-Hilbert space-filling curve. This is a one-dimensional object filling up the three-dimensional space. An example of such domain decomposition is shown in figure 2. This strategy is known to be the optimal choice if one considers the rather large ensemble of all possible AMR grids. In some cases, however, it is no longer an efficient strategy. The planar decomposition, for example, sets up computational domains according to one coordinate only (the altitude z for example). Each processor receives a layer of cells whose thickness is automatically adjusted in order to optimize load balance. The angular decomposition follows the same strategy, except that now the coordinate is the polar angle around a given axis in the simulation box. These various orderings can be adapted easily to account for specific constraints. The user is encouraged to edit and modify the routine `load_balance.f90` in directory `amr/`.

In case of parallel execution, RAMSES performs hard disk outputs in a very simple way: each processor creates its own file. Therefore, in directory `output_00001/`, one can find several files with a numbering corresponding to the processor number. One should bear this in mind when using the snapshots generated by RAMSES.

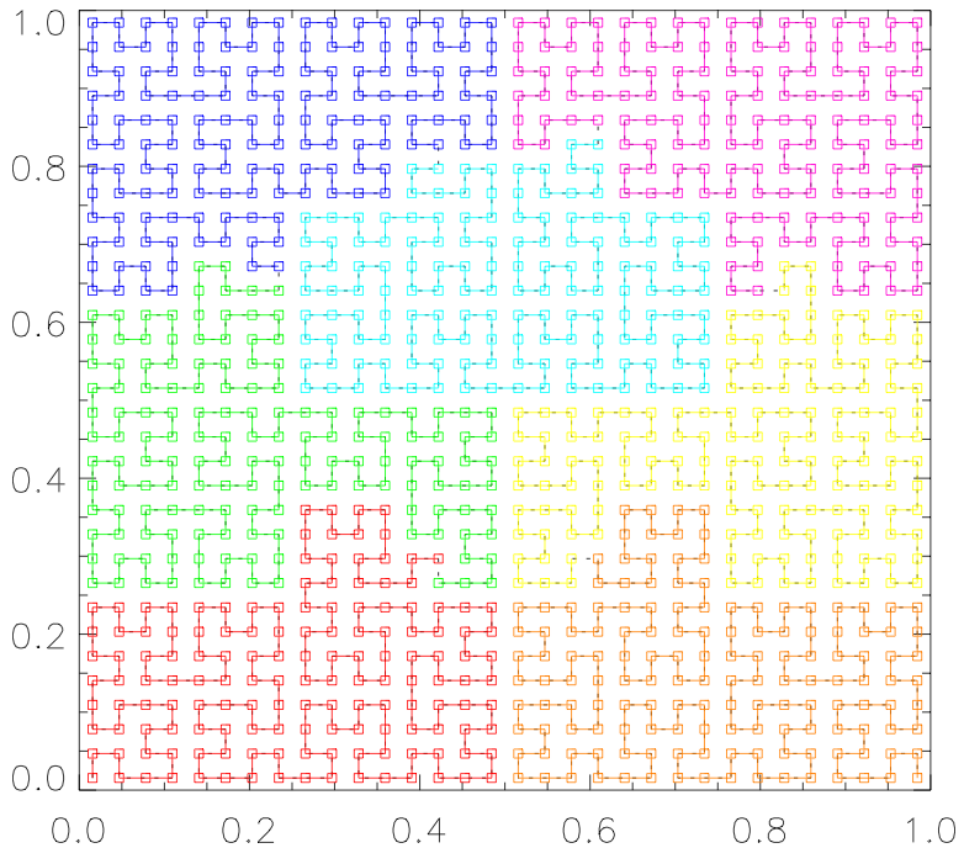


Figure 2: Domain decomposition of the unit square for a 32^2 grid over 7 processors using the Peano-Hilbert space-filling curve shown as the continuous line.

4.5 Post-processing utilities

Several post-processing codes are available in the current package in directory `utils/f90`. These are very simple pieces of software performing rather basic operations on RAMSES generated outputs. Users are encouraged to edit and modify these routines in order to design specific post-processing applications. We briefly describe them now.

- **amr2map**: this application reads RAMSES outputs and generates a projected map along one principal axis. The output is a binary Fortran image that can be read using any image-processing tool.
- **amr2cube**: this application reads RAMSES outputs and generates a 3D Cartesian cube with only one flow variable represented. The output is a binary Fortran file (raw data or graphic data) or a VTK file that can be read using any data visualization tool.
- **part2map**: this application reads RAMSES outputs for collisionless particles only and projected the particle distribution along one principal axis. The output is a binary Fortran image.
- **part2cube**: this application reads RAMSES outputs for particles only and generates a CIC interpolated density field. The output is a binary Fortran file.

Each one of these codes is a stand-alone application that can be compiled straightforwardly by typing directly for example:

```
$ f90 amr2map.f90 -o amr2map
```

In directory `utils/idl`, you can find a set of IDL routines to be used in order to display AMR data using IDL (see <http://www.ittviz.com/>). Here is an example of interactive commands you need to type within your IDL session to watch RAMSES data (for example using `sedov2d.nml`).

```
IDL> rd_amr,a,nout=2 ; Read AMR data from snapshot nr 2 and store in a
IDL> rd_hydro,h,nout=2 ; Read hydro data and store in variable h
IDL> window,0,xs=512,ys=512 ; Set up a square window
IDL> loadct,33 ; Load a nice color table
IDL> tv2d,a,h,type=1,/log,/show ; Plot a density map with the grid
```

Here is another example to plot a density profile from the previous data.

```
IDL> amr2cell,a,h,c ; Convert AMR data into cell-based data
IDL> r=sqrt(c.x^2+c.y^2) ; Compute cell radius
IDL> d=c.var(*,0) ; Compute cell density
IDL> plot,r,d,psym=6
```

For 3D data, you can use a simple raytracing algorithm to project various quantities along one of the box principal axis.

```
IDL> ray3d,a,h,lmin=7,lmax=9,/zproj,/ave,type=1,/log
; Project the average density along the z-axis
```

4.6 Zoom simulations

Another interesting cosmological application for RAMSES is the so-called “zoom” technology or “re-simulation” process. Let us consider the large-scale periodic box simulation we have presented in the previous section, performed with 128^3 particles by RAMSES with the `grafic` files stored in directory `/scratchdir/grafic_files`. After using the `sod` application, all dark matter halos in the final output have been identified. One halo is believed to be a good candidate for re-simulation. A quasi-spherical region must be determined, whose size and position are optimized in order to contain all the particles ending up in the final halo. A new set of initial conditions must then be generated using `mpgrafic`, providing the same large-scale modes than the previous run, but allowing now to simulate up to 1024^3 particles. A suite of applications is available in directory `utils/f90` to perform the extraction of a high-resolution region containing the halo particles only. These codes are called `center_grafic`, `extract_grafic` and `degrade_grafic`. The idea is to center the simulation box on the high-resolution region, to extract a nested collection of rectangular grids around this position with various particle masses. The initial conditions data for each box are stored in a different directory. The original data centered on the region center can be called `boxlen100_n256`, where 100 stands for the box size in h^{-1} Mpc and 128 for the number of particles per box length. The nested box hierarchy we obtained using our various utilities is now:

```
boxlen100_n128/  
boxlen50_n128/  
boxlen25_n128/  
boxlen12p5_n128/
```

Each of these directories should contain 7 `grafic` files. These names should be now inserted in the Parameter File, in the `&INIT_PARAMS` block, as

```
&INIT_PARAMS  
filetype='grafic'  
initfile(1)='/scratchdir/grafic_directories/boxlen100_n128'  
initfile(2)='/scratchdir/grafic_directories/boxlen50_n128'  
initfile(3)='/scratchdir/grafic_directories/boxlen25_n128'  
initfile(4)='/scratchdir/grafic_directories/boxlen12p5_n128'  
/
```

The re-simulation is now ready to go. Those are our last words on cosmological simulations and how to run them using only Parameter Files as Runtime Parameters. We now describe how to use RAMSES with more advanced settings.

5 Advanced simulations

For truly innovative scientific applications, the user is usually forced to define complex initial conditions, to impose time varying boundary conditions or to use more than the 5 standard Euler variables (chemical species for example). We briefly describe the easiest way to do it in RAMSES.

5.1 Patching the code

The general philosophy to design advanced RAMSES applications is to “patch the code”. What do we mean by that? A few key routines have been designed in RAMSES in a user-friendly fashion, allowing the user to edit the file, modify it according to its needs and recompile the code. For that, it is recommended to create your own directory, for example `mypatch/`, in which you will copy the various files you plan to modify. In the `Makefile`, you need to specify the complete path of this directory in the `PATCH` variable, as:

```
PATCH=/home/foo/mypatch
```

The `make` command will seek for source files in this directory first, compile and link them if present. If not, it will use the default source files already available in the RAMSES package. Virtually any RAMSES source file can be modified to obtain a “patched” version of RAMSES that fulfill your needs. Usually, however, only a few routines need to be modified in order to perform advanced simulations. These routines will be described now in more detail. The corresponding files are stored in various RAMSES subdirectories. These are: `amr/units.f90`, `hydro/boundana.f90`, `hydro/condinit.f90`, `poisson/gravana.f90`, `poisson/rho_ana.f90`, `hydro/cooling_fine.f90`. Of course, other routines of RAMSES can be modified at will, although potential changes might be more complicated to implement. A simple example of patch can be found in the directory `patch/` of the package.

5.2 Physical units

This very simple routine can be found in directory `amr/` and is called `units.f90`. It is used to set the conversion factors from the user units into the cgs unit system. In this routine, the user must provide 3 scaling factors, namely `scale_d` for the density units in g.cm^{-3} , `scale_l` for the length scale in cm and `scale_t` for the time scale in seconds. For self-gravity runs, since RAMSES assumes $G = 1$ in the Poisson equation, it is mandatory to define the time scale as `scale_t=1.0/sqrt(G*scale_d)` with $G=6.67\text{d-8}$. These scaling factors are stored in the output files and can be used later on during post-processing.

5.3 Initial conditions

This routine can be found in directory `hydro/` and is called `condinit.f90`. It is self-documented. The calling sequence is just `call condinit(x,u,dx,ncell)`, where `x` is an input array of cell center positions, `u` is an output array containing the volume average of the fluid conservative variables, namely $(\rho, \rho u, \rho v, \rho w$ and $E)$, in this exact order. If more variables are defined, using the `-DNVAR` directive, then the user should exploit this routine to define them too. `dx` is a single real value containing the cell size for all the cells and `ncell` is the number of cells. This routine can be used to set the initial conditions directly with Fortran instructions. Examples of such instructions can be found in directory `patch/`.

Another way to define initial conditions in RAMSES is by using input files. For the hydro solver, these files are always in the `grafic` format. We have already explained how to use the `grafic` format for cosmological runs. For non-cosmological runs, initial conditions can

be defined using the exact same format, except that instead of 4 files (`ic_deltab`, `ic_velbx`, `ic_velby` and `ic_velbz`), one now needs 5 files called (`ic_d`, `ic_u`, `ic_v`, `ic_w` and `ic_p`) and containing the fluid primitive variables.

For collisionless particles, the `grafic` format is used only for cosmological runs, for which particles are initially placed at the cell centers of a Cartesian grid. For non-cosmological runs, the particles' initial positions, velocities and masses are read in an ASCII file, in which each line corresponds to one particle, and should contain the following particle attributes: `x,y,z,vx,vy,vz,m`.

5.4 Boundary conditions

As explained in the previous sections, RAMSES can provide boundary conditions of different types: periodic (default mode), reflexive, outflow and imposed. This is performed in RAMSES using ghost regions, in which the fluid variables are set in order to obtain the required boundary. Ghost regions are defined in the namelist block `&BOUNDARY_PARAMS`. Each region is identified by its position, its type and eventually by the value of the fluid variables.



The exact order with which boundary regions are entered in the namelist block is very important. Let us consider the 4 boundary regions shown in figure 3. They are defined by the following namelist block:

```
&BOUNDARY_PARAMS
nboundary=4
ibound_min=-1,+1,-1,-1
ibound_max=-1,+1,+1,+1
jbound_min= 0, 0,+1,-1
jbound_max= 0, 0,+1,-1
bound_type= 1, 1, 1, 1
/
```

The first region is located in the rectangle defined by coordinate ($i = -1, j = 0$), while the third region is defined by coordinates ($-1 \leq i \leq +1, j = +1$). The boundary type for all 4 regions is set to “reflexive” (`bound_type=1`). The fluid variables within the ghost region are therefore taken equal to the values of their symmetric cells, with respect to the boundary. This is why the order of the ghost regions is so important: regions 1 and 2 are updated first, using only the fluid variables within the computational domain. Regions 3 and 4 are updated afterwards, using the fluid variables within the computational domain, but also within regions 1 and 2. In this way, all cells within boundary regions are properly defined, especially in the 4 corners of the computational domain.

It is possible to define only 2 regions (say regions 1 and 2 in figure 3), the orthogonal direction will be considered as periodic. For gravity runs, the gravitational force is also updated in the ghost regions, following the same rules as the velocity vector.

For the Poisson equation, however, boundary conditions are either periodic, if no ghost regions are defined in the corresponding direction, or “ $\phi = 0$ ” Dirichlet boundary conditions within ghost regions. No other types of boundary conditions for the Poisson equation have been implemented (such as isolated, reflexive and so on).

If `bound_type=3`, boundary conditions must be imposed by the user. The first possibility is to use parameters `d_bound`, `u_bound...` to set a constant fluid state within the desired

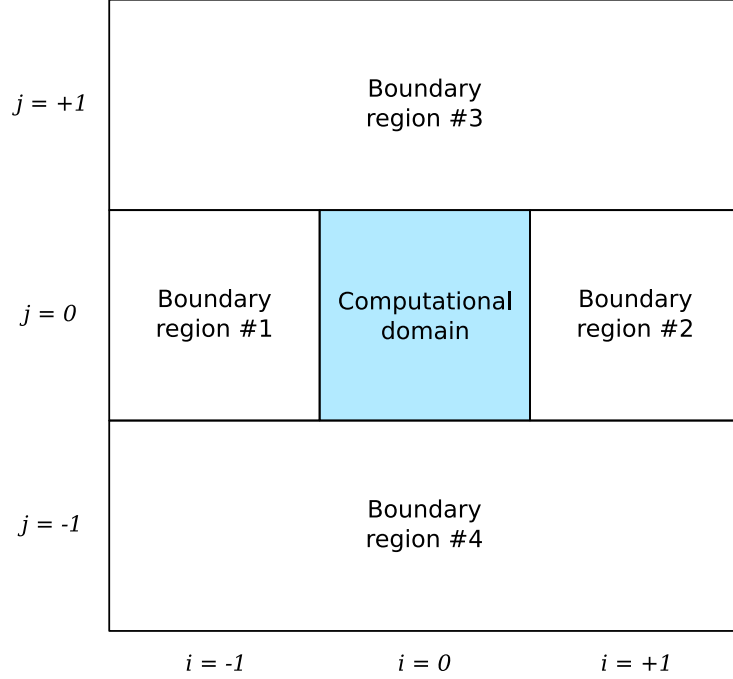


Figure 3: Example of ghost regions used in RAMSES to impose specific boundary conditions.

ghost region. For more advanced applications, the user is encouraged to patch the routine `boundana.f90` within directory `hydro/`. This routine is very similar to routine `condinit.f90`. The calling sequence is `call boundana(x,u,dx,ibound,ncell)`. The ghost region number is therefore provided, so the user can specify some region-dependent fluid conditions.

5.5 External gravity sources

If `bound_type=3`, boundary conditions must be imposed also for the gravitational force. This is performed by modifying routine `gravana.f90` in directory `poisson/`. If `gravity_type>0`, this routine is also used to specify the gravitational force within the computational domain. Note that in this case, the fluid is not self-gravitating anymore. There is another way to impose an external gravity source and in the same time, to solve for the Poisson equation. This is done using routine `rho_ana.f90` in directory `poisson/`. In this routine, again very similar to the previously presented Fortran routines, the user can specify the density profile for the external gravity source. This density profile will be added to the fluid density as the source term in the Poisson equation.

5.6 External thermal sources

The final routine that can be easily modified by the user is `cooling_fine.f90` in directory `hydro/`. This routine is used if `cooling=.true.` or if the polytropic temperature `T2_star>0.0`. In the first case, cooling and heating source terms are added to the energy equation and solved by a fully implicit integration scheme. In the second case, the thermal energy of the fluid is not allowed to be lower than a polytropic Equation-Of-State, for which one has $T/\mu = (T/\mu)_*(\rho/\rho_*)^{\gamma_*-1}$. All starred parameters can be set within the namelist block `&PHYSICS_PARAMS`. On the other hand, the user might want to modify routine `cooling_fine.f90` in order to implement more complex thermal modeling of the fluid.

6 Publication policy

The RAMSES code is freely available for non-commercial use under the CeCILL License agreement. If a paper is published with simulations performed with RAMSES, the authors should cite the following paper, in which the RAMSES code was presented for the first time:

Teyssier, Romain, “Cosmological hydrodynamics with Adaptive Mesh Refinement: a new high-resolution code called RAMSES”, Astronomy and Astrophysics, 2002, volume 385, page 337

If the same users need some basic help from the author on how to use RAMSES, or if the simulations performed have needed from the code’s author some small adaptation of the code, a small sentence like “We thank Romain Teyssier for...” in the Acknowledgment section will do it.

If, on the other hand, the simulations performed requires the code’s author to be more deeply involved in the project (new developments, new simulations from the author’s side), then co-authorship of the paper is asked.

Index

makefile options and flags

F90 option, 6
FFLAGS option, 6
NDIM option, 6, 9
NPRES option, 6, 25
NVAR option, 6, 21, 30
NVECTOR option, 6
PATCH option, 30
WITHOUTMPI option, 6, 26

namelist blocks

&AMR_PARAMS block, 12, 15
&BOUNDARY_PARAMS block, 12, 19, 31
&CLUMPFIND_PARAMS block, 13
&COOLING_PARAMS block, 13
&FEEDBACK_PARAMS block, 13
&HYDRO_PARAMS block, 12, 20
&INIT_PARAMS block, 12, 16, 24, 29
&MOVIE_PARAMS block, 18
&OUTPUT_PARAMS block, 12, 17
&PHYSICS_PARAMS block, 12, 13, 21, 32
&POISSON_PARAMS block, 12, 22
&REFINE_PARAMS block, 12, 15, 23
&RUN_PARAMS block, 12, 14, 24, 26
&SF_PARAMS block, 13
&SINK_PARAMS block, 13
&UNITS_PARAMS block, 13

namelist parameters & log entries

0, 18
1, 18
2, 18
3, 18
4, 18
5, 18
6, 18
7..., 18
J21, 21
T2_star, 21, 32
a_refine, 23
a_spec, 21
aexp_ini, 16
angular, 14
aout, 17
a log entry, 9
b_refine, 23
bound_type, 19, 31, 32
boxlen, 15, 24
cg_levelmin, 22

cooling, 21, 32
cosmo, 14, 24
courant_factor, 20
d_bound, 19, 31
d_region, 16
del_star, 21
deltax_frame, 18
deltay_frame, 18
deltaz_frame, 18
econs log entry, 9
epsilon, 22
err_grad_d, 23
err_grad_p, 23
err_grad_u, 23
eta_sn, 21
exp_refine, 23
exp_region, 16
filetype, 16, 24
floor_d, 23
floor_p, 23
floor_u, 23
foutput, 17, 26
g_star, 21
gamma, 20
gravity_params, 22
gravity_type, 22
haardt_madau, 21
hilbert, 14
hydro, 14
ibound_max, 19
ibound_min, 19
imovout, 18
imov, 18
initfile, 16, 24
interpol_type, 23
interpol_var, 23
jbound_max, 19
jbound_min, 19
jeans_refine, 23
kbound_max, 19
kbound_min, 19
length_x, 16
length_y, 16
length_z, 16
levelmax_frame, 18
levelmax, 9, 14, 15, 23
levelmin, 8, 14, 15, 23, 24
m_refine, 23

- mass_sph, 23
- mcons log entry, 9
- metal, 21
- movie_vars, 18
- movie, 18
- multiple, 16
- n_star, 21
- nboundary, 19
- ncontrol, 14
- nexpand, 15
- ngridmax, 15, 25
- ngridtot, 15, 25
- nh_frame, 18
- niter_riemann, 20
- noutput, 17
- npartmax, 15, 25
- nparttot, 15, 25
- nregion, 16
- nremap, 14, 26
- nrestart, 14, 26
- nstepmax, 14
- nsubcycle, 14
- nw_frame, 18
- ordering, 14, 26
- p_bound, 19
- p_region, 16
- pic, 14, 22
- planar, 14
- poisson, 14, 22
- pressure_fix, 20
- proj_axis, 18
- r_refine, 23
- region_type, 16
- riemann2d, 20
- riemann, 20
- slope_type, 20
- smallc, 20
- smallr, 20
- t_star, 21
- tendmov, 18
- tout, 17
- u_bound, 19, 31
- u_region, 16
- v_bound, 19
- v_region, 16
- verbose, 14
- w_bound, 19
- w_region, 16
- x_center, 16
- x_refine, 23
- xcentre_frame, 18

- y_center, 16
- y_refine, 23
- ycentre_frame, 18
- yield, 21
- z_ave, 21
- z_center, 16
- z_refine, 23
- zcentre_frame, 18

solvers

- acoustic Riemann solver, 20
- exact Riemann solver, 20
- hllc Riemann solver, 20
- hlld Riemann solver, 20
- hll Riemann solver, 20
- hydro Riemann solver, 20
- hydro solver, 6
- llf Riemann solver, 20
- mhd solver, 6
- multimat solver, 6
- rhd solver, 6
- roe Riemann solver, 20
- upwind Riemann solver, 20

utilities & directories

- amr/ directory, 6, 24, 26, 30
- amr2cube utility, 28
- amr2map utility, 28
- bin/ directory, 6
- center_grafic utility, 29
- degrade_grafic utility, 29
- extract_grafic utility, 29
- grafic2 package, 24
- grafic package, 29–31
- hydro/ directory, 6, 30, 32
- mpgrafic package, 4, 24, 29
- namelist/ directory, 7, 9, 19, 24
- output_00001/ directory, 26
- part2cube utility, 28
- part2map utility, 28
- patch/ directory, 30
- poisson/ directory, 32
- sod utility, 29
- trunk/ramses/ directory, 6
- utils/f90 directory, 28, 29
- utils/idl directory, 28